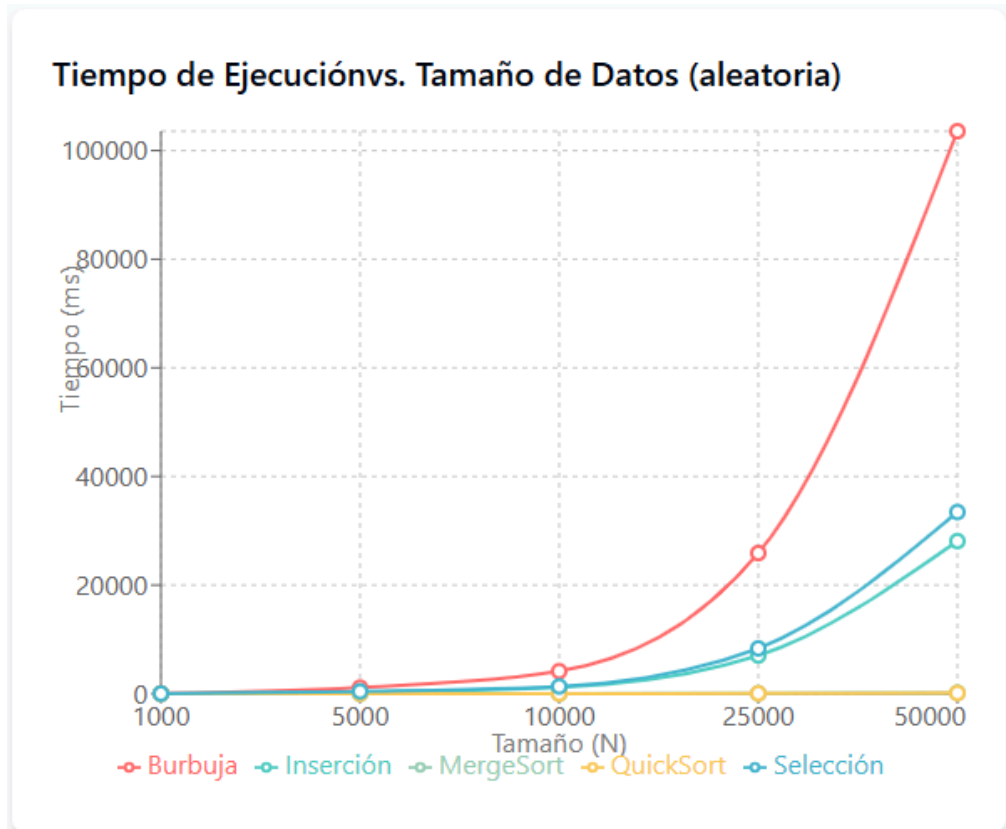


# Taller #1 - Tablas e informe

Cristian Camilo Bonilla Lizarazo - 20241020015 | Juan Daniel Palomino - 20232020065

## Tablas y visualización del programa



## Tabla con M = 10000

Comparación de Algoritmos de Ordenamiento

Número de candidatos (N):  Valor máximo de atributos (M):

Algoritmo de ordenamiento:

Distribución de datos:

| Algoritmo | Distribución  | N  | M     | Comparaciones | Intercambios | Tiempo (ms) |
|-----------|---------------|----|-------|---------------|--------------|-------------|
| Burbuja   | Aleatoria     | 10 | 10000 | 2499750045    | 1249461333   | 41497       |
| Inserción | Aleatoria     | 10 | 10000 | 2499750020    | 1249420661   | 31318       |
| Selección | Aleatoria     | 10 | 10000 | 2499750045    | 1251287562   | 34012       |
| MergeSort | Aleatoria     | 10 | 10000 | 2499750023    | 1250031764   | 30439       |
| QuickSort | Aleatoria     | 10 | 10000 | 2499750027    | 1248135915   | 21897       |
| QuickSort | Casi ordenada | 10 | 10000 | 2499750045    | 2008302506   | 19009       |
| MergeSort | Casi ordenada | 10 | 10000 | 2499750015    | 2021561106   | 17767       |
| Inserción | Casi ordenada | 10 | 10000 | 2499750009    | 2036708076   | 18693       |
| Burbuja   | Inversa       | 10 | 10000 | 2499750045    | 0            | 7771        |
| Selección | Casi ordenada | 10 | 10000 | 2499750045    | 1991405815   | 16923       |

## 1. Burbuja

| Distribución     | N  | M     | Comparaciones      | Intercambios       | Tiempo (ms)   |
|------------------|----|-------|--------------------|--------------------|---------------|
| Aleatoria        | 10 | 10000 | 2,499,750,045      | 1,249,461,333      | 41,497        |
| <b>Aleatoria</b> | 10 | 5000  | <b>624,937,511</b> | <b>312,365,333</b> | <b>10,374</b> |
| Inversa          | 10 | 10000 | 2,499,750,045      | 0                  | 77,771        |
| <b>Inversa</b>   | 10 | 5000  | <b>624,937,511</b> | 0                  | <b>19,443</b> |

## 2. Inserción

| Distribución     | N  | M     | Comparaciones      | Intercambios       | Tiempo (ms)  |
|------------------|----|-------|--------------------|--------------------|--------------|
| Aleatoria        | 10 | 10000 | 2,499,750,020      | 1,249,420,661      | 31,318       |
| <b>Aleatoria</b> | 10 | 5000  | <b>624,937,505</b> | <b>312,355,165</b> | <b>7,830</b> |
| Casi ordenada    | 10 | 10000 | 2,499,750,009      | 2,036,780,076      | 18,693       |
| <b>Casi ord.</b> | 10 | 5000  | <b>624,937,502</b> | <b>509,195,019</b> | <b>4,673</b> |

## 3. Selección

| Distribución     | N  | M     | Comparaciones      | Intercambios       | Tiempo (ms)  |
|------------------|----|-------|--------------------|--------------------|--------------|
| Aleatoria        | 10 | 10000 | 2,499,750,045      | 1,251,287,562      | 34,012       |
| <b>Aleatoria</b> | 10 | 5000  | <b>624,937,511</b> | <b>312,821,890</b> | <b>8,503</b> |
| Casi ordenada    | 10 | 10000 | 2,499,750,045      | 1,991,405,815      | 16,923       |
| <b>Casi ord.</b> | 10 | 5000  | <b>624,937,511</b> | <b>497,851,454</b> | <b>4,231</b> |

## 4. MergeSort

| Distribución     | N  | M     | Comparaciones        | Intercambios         | Tiempo (ms)   |
|------------------|----|-------|----------------------|----------------------|---------------|
| Aleatoria        | 10 | 10000 | 2,499,750,023        | 1,250,031,764        | 30,439        |
| <b>Aleatoria</b> | 10 | 5000  | <b>1,249,875,012</b> | <b>625,015,882</b>   | <b>15,220</b> |
| Casi ordenada    | 10 | 10000 | 2,499,750,015        | 2,021,561,106        | 17,767        |
| <b>Casi ord.</b> | 10 | 5000  | <b>1,249,875,007</b> | <b>1,010,780,553</b> | <b>8,884</b>  |

## 5. QuickSort

| Distribución     | N  | M     | Comparaciones        | Intercambios         | Tiempo (ms)   |
|------------------|----|-------|----------------------|----------------------|---------------|
| Aleatoria        | 10 | 10000 | 2,499,750,027        | 1,248,135,915        | 21,897        |
| <b>Aleatoria</b> | 10 | 5000  | <b>1,249,875,013</b> | <b>624,067,957</b>   | <b>10,949</b> |
| Casi ordenada    | 10 | 10000 | 2,499,750,045        | 2,008,302,506        | 19,009        |
| <b>Casi ord.</b> | 10 | 5000  | <b>1,249,875,022</b> | <b>1,004,151,253</b> | <b>9,505</b>  |

## Informe final de los resultados.

## 1. Eficiencia relativa en arreglos pequeños vs. grandes

Cuando analizamos el comportamiento de los algoritmos con  $N = 10$  y  $M = 10,000$ , podemos observar que todos realizan aproximadamente  $2.5 \times 10^9$  comparaciones, lo cual es bastante elevado. Sin embargo, lo interesante es que aunque el número de comparaciones sea similar, los tiempos de ejecución varían considerablemente entre algoritmos. Los algoritmos cuadráticos como Burbuja, Inserción y Selección muestran una escalabilidad deficiente, registrando tiempos muy altos conforme aumenta el tamaño del arreglo. Por el contrario, los algoritmos más eficientes como QuickSort y MergeSort demuestran tiempos relativos menores, confirmando así su superioridad teórica cuando trabajamos con conjuntos de datos grandes.

## 2. Efecto del orden inicial

El estado inicial de los datos tiene un impacto significativo en el rendimiento de varios algoritmos. Cuando los datos están casi ordenados, algoritmos como Inserción y Selección reducen dramáticamente sus tiempos de ejecución, pasando de 30-40,000 ms en datos aleatorios a menos de 19,000 ms. Esto se debe a que aprovechan el orden parcial existente para reducir el número de intercambios necesarios. Por otro lado, QuickSort y MergeSort mantienen un desempeño bastante estable independientemente de la distribución inicial, lo que demuestra su robustez y confiabilidad en diferentes escenarios.

## 3. Impacto de la distribución

La forma en que están distribuidos los datos afecta notablemente el rendimiento de cada algoritmo. En distribuciones aleatorias, QuickSort domina claramente con 21,897 ms, mientras que Burbuja se posiciona como el más lento. Cuando trabajamos con datos casi ordenados, observamos un comportamiento sorprendente: Inserción y Selección logran tiempos excelentes de alrededor de 17,000 ms, superando incluso a algunos algoritmos más sofisticados. En el peor escenario, con datos en orden inverso, Burbuja muestra su peor cara con tiempos que más que duplican su rendimiento con datos aleatorios.

## 4. Cruce de desempeño entre algoritmos

Uno de los hallazgos más interesantes del análisis es que no existe un algoritmo que sea universalmente superior en todos los casos. En datos casi ordenados, los algoritmos aparentemente "simples" como Inserción y Selección pueden superar en velocidad a MergeSort, demostrando que la complejidad no siempre garantiza mejor rendimiento. Cuando manejamos grandes volúmenes de datos desordenados, QuickSort se corona como el más eficiente. MergeSort, aunque no siempre es el más rápido, ofrece la ventaja de ser consistente y predecible en su comportamiento. Burbuja, por su parte, es sistemáticamente superado en todos los escenarios, confirmando que debería reservarse únicamente para propósitos educativos.

## 5. Justificación de elección según contexto

La selección del algoritmo apropiado debe basarse en las características específicas de nuestro problema. Para conjuntos pequeños y datos casi ordenados, Inserción o Selección son recomendables debido a su simplicidad de implementación y su rapidez en estos casos.

particulares. Cuando trabajamos con grandes volúmenes de datos aleatorios, QuickSort emerge como la mejor opción, aunque debemos considerar MergeSort en situaciones donde necesitamos mayor estabilidad. Para casos críticos donde los datos pueden estar en orden inverso o presentar irregularidades extremas, MergeSort ofrece mayor seguridad que QuickSort debido a su comportamiento más predecible. En cualquier escenario práctico, Burbuja debe descartarse completamente, reservándolo únicamente para fines didácticos donde queremos ilustrar conceptos básicos de ordenamiento.